

Techniki Mikroprocesorowe

Sprawozdanie – Laboratorium 6

Temat: Wyświetlacz LCD (Sterownik HD44780).

Autor: Kamil Sitarski

Numer albumu: [REDACTED]

Grupa dziekańska: [REDACTED]

Data wykonania ćwiczenia: 2.06.2026r.

1. Opis zadania

Celem zadania laboratoryjnego było zaprojektowanie, zaimplementowanie oraz uruchomienie programu w języku C, umożliwiającego niskopoziomowe sterowanie wyświetlaczem LCD (opartym na sterowniku jednoukładowym HD44780) za pomocą mikrokontrolera AVR ATmega32 taktowanego wewnętrznym sygnałem zegarowym o częstotliwości 1 MHz.

Zadanie polegało na wykorzystaniu trybu 4-bitowego w celu optymalizacji liczby wykorzystywanych linii sygnałowych mikrokontrolera. Zamiast standardowej magistrali 8-bitowej, transmisja danych została zrealizowana przy użyciu jedynie 4 linii danych (podłączonych do starszych bitów portu PORTA) oraz 2 linii sterujących. Głównym celem programu było poprawne przeprowadzenie programowej procedury inicjalizacji wyświetlacza LCD, programowe pozycjonowanie kursora w pamięci DDRAM (Display Data RAM), a następnie sekwencyjne wyświetlenie długiego tekstu statycznego: "Kamil Sitarski 123456". Dodatkowo algorytm musiał uwzględniać automatyczne zawijanie tekstu do drugiego wiersza po osiągnięciu krawędzi ekranu (16. kolumny).

2. Opis konfiguracji sprzętowej

Zgodnie z dyrektywami preprocesora zawartymi w kodzie źródłowym, do współpracy z wyświetlaczem LCD przeznaczono **PORTA** mikrokontrolera ATmega32. Przypisanie linii sygnałowych zdefiniowano następująco:

- **Magistrala danych (D4–D7):** Podłączona do starszego półportu PORTA, tj. pinów **PA4, PA5, PA6, PA7**. Dane przesyłane są w postaci paczek 4-bitowych.
- **Linia strobująca EN (Enable):** Podłączona do pinu **PA0 (PIN_E)**. Generuje impulsy zegarowe zatwierdzające operacje zapisu.
- **Linia wyboru rejestru RS (Register Select):** Podłączona do pinu **PA1 (PIN_RS)**. Odpowiada za przełączanie trybu pracy sterownika pomiędzy rejestrem instrukcji (RS = 0) a rejestrem danych tekstowych (RS = 1).
- *Uwaga systemowa:* Linia zapisu/odczytu R/W wyświetlacza w tej konfiguracji zostaje na stałe zwarta do potencjału masy (GND), co konfiguruje moduł LCD wyłącznie do przyjmowania danych (tryb zapisu).

3. Opis kodu

Poniżej przedstawiono analizę strukturalną programu z podziałem na poszczególne bloki funkcjonalne i bezpośrednim odniesieniem do kodu źródłowego.

- **Definicje preprocesora i dyrektywy wejściowe**

```
#define F_CPU 1000000UL
#define PORT_LCD PORTA
#define PIN_E PA0
#define PIN_RS PA1
```

```
#include <avr/io.h>
#include <util/delay.h>
#include <string.h>
```

Na początku programu zdefiniowano częstotliwość taktowania rdzenia mikrokontrolera jako 1 MHz (1000000UL), co jest niezbędne do prawidłowego wyliczania czasu przez makra opóźniające z biblioteki <util/delay.h>. Makra PORT_LCD, PIN_E oraz PIN_RS przypisują fizyczne piny procesora (PA0, PA1) do ich ról w sterowaniu ekranem, co ułatwia ewentualną późniejszą zmianę portu w kodzie.

- **Funkcja transmisji połówki bajtu**

```
void LCDwypiszStarszaCzesc(char bajt){
    PORT_LCD |= (1 << PIN_E);
    _delay_us(50);
    PORT_LCD = (bajt & 0xF0) | (PORT_LCD & 0x0F);
    _delay_us(50);
    PORT_LCD &= ~(1 << PIN_E);
}
```

Jest to podstawowa funkcja manipulacji sprzętowej. Odpowiada za wystawienie 4 bitów na linii danych. Operacja (bajt & 0xF0) wycina starszą część przesyłanej wiadomości, a (PORT_LCD & 0x0F) zapobiega zniszczeniu aktualnego stanu młodszych pinów portu (gdzie znajdują się linie sterujące E i RS). Synchronizacja i zatwierdzenie danych w kontrolerze HD44780 następuje poprzez wytworzenie zbocza opadającego na pinie PIN_E (sekwencyjne ustawienie stanu wysokiego, odczekanie 50 μ s i powrót do stanu niskiego).

- **Funkcja transmisji pełnego bajtu w trybie 4-bitowym**

```
void LCDwypisz(char bajt){
    LCDwypiszStarszaCzesc(bajt);
    _delay_us(50);
    PORT_LCD |= (1 << PIN_E);
    _delay_us(50);
    PORT_LCD = ((bajt & 0x0F) << 4) | (PORT_LCD & 0x0F);
    _delay_us(50);
    PORT_LCD &= ~(1 << PIN_E);
}
```

Ponieważ sterownik pracuje w trybie 4-bitowym, każdy 8-bitowy znak lub rozkaz musi zostać podzielony na dwie paczki. Funkcja najpierw wywołuje omówioną wyżej procedurę `LCDwypiszStarszaCzesc(bajt)`, która wysyła bity 7–4. Następnie przygotowuje port do wysłania młodszej części: instrukcja `((bajt & 0x0F) << 4)` pobiera bity 3–0 i przesuwa je komórkowo na pozycje starsze (tak, aby pojawiły się na wyprowadzeniach PA4–PA7). Całość procesu jest ponownie zatwierdzana impulsem strobuującym na linii `PIN_E`.

- **Funkcja zapisu rozkazów konfiguracyjnych**

```
void LCDinstrukcja(char bajt){
    PORT_LCD &= ~(1 << PIN_RS);
    _delay_ms(1);
    LCDwypisz(bajt);
    _delay_ms(1);
    PORT_LCD |= (1 << PIN_RS);
}
```

Służy do wpisywania poleceń niebędących tekstem (np. czyszczenie ekranu, włączanie mrugania). Realizuje to poprzez jawne wyzerowanie linii wyboru rejestru (`PORT_LCD &= ~(1 << PIN_RS)`). Przy stanie $RS = 0$ kontroler HD44780 interpretuje nadchodzące bajty jako instrukcje. Po przestaniu bajtu za pomocą `LCDwypisz()`, linia `RS` jest asekuracyjnie podnoszona z powrotem do stanu wysokiego.

- **Funkcja pozycjonowania kursora w pamięci DDRAM**

```
void LCDustawKursor(uint8_t w, uint8_t k){
    PORT_LCD &= ~(1 << PIN_RS);
    uint8_t bajt = 0b10000000;
    if(w==1){
        bajt = 0b11000000;
    }
    bajt += k;
    LCDwpisz(bajt);
    _delay_ms(1);
    PORT_LCD |= (1 << PIN_RS);
}
```

Funkcja pozwala na wskazanie miejsca na ekranie, w którym rozpocznie się zapis tekstu. Adresowanie pamięci DDRAM w sterowniku zaczyna się od komendy binarnej **0b10000000** (0x80) dla pierwszego wiersza (indeksowanego w tym kodzie jako **w = 0**). Jeżeli parametr wiersza wynosi 1 (**if(w==1){...}**), bazowa komenda przyjmuje wartość **0b11000000** (0xC0), co odpowiada adresowi startowemu drugiej linii wyświetlacza. Wartość kolumny **k** jest dodawana matematycznie do adresu bazowego przed wysłaniem rozkazu.

- **Funkcja wyświetlania łańcuchów znakowych z kontrolą przepiętnienia linii**

```
void LCDtekst(char *tekst, uint8_t kolumna){
    for(uint8_t i=0 ; i<strlen(tekst);i++){
        if(kolumna==16){ LCDustawKursor(1,0); }
        _delay_ms(1);
        LCDwpisz(tekst[i]);
        kolumna++;
    }
}
```

Odpowiada za sekwencyjne przetwarzanie tablicy znaków przekazanej przez wskaźnik ***tekst**. Pętla wykonuje się tyle razy, ile wynosi długość ciągu obliczona funkcją **strlen()**. Kluczowym elementem algorytmu jest instrukcja warunkowa **if(kolumna==16)**. Ponieważ wyświetlacz alfanumeryczny ma ograniczoną szerokość do 16 kafelków w jednym wierszu, osiągnięcie tej wartości powoduje automatyczne wywołanie funkcji **LCDustawKursor(1, 0)**. Gwarantuje to płynne, programowe przeniesienie nadmiarowego tekstu na początek dolnej linii (wiersz 1, kolumna 0).

- **Procedura inicjalizacyjna wyświetlacza LCD**

```
void LCD_inicjalizacja(){
    DDRA = 0xFF;

    LCDwypiszStarszaCzesc(0x30);
    _delay_ms(5);
    LCDwypiszStarszaCzesc(0x30);
    _delay_ms(1);
    LCDwypiszStarszaCzesc(0x30);
    _delay_ms(1);
    LCDwypiszStarszaCzesc(0x20);

    _delay_ms(1);
    LCDwypisz(0x28);
    _delay_ms(1);
    LCDwypisz(0b00001100); //00001100
    _delay_ms(1);
    LCDwypisz(0x06);
    _delay_ms(1);
    LCDwypisz(0x01);
    _delay_ms(1);
    PORT_LCD |= (1 << PIN_RS);
}
```

Pierwsza linijka `DDRA = 0xFF` ustawia cały Port A jako wyjścia cyfrowe. Następnie funkcja wykonuje ściągę, opisaną w dokumentacji HD44780 sekwencję resetu programowego dla trybu 4-bitowego (trzykrotny zapis wartości `0x30` z przerwami czasowymi), po czym wysyła potówkę `0x20` inicjującą magistralę 4-bitową. Druga faza to wystanie kompletnych bajtów konfiguracyjnych:

- `0x28` – aktywacja pracy 2-wierszowej z matrycą znaków 5x8 punktów.
- `0b00001100` – włączenie wyświetlacza, wyłączenie widoczności kursora oraz jego mrugania.
- `0x06` – włączenie automatycznego przesuwania kursora w prawo po wpisaniu znaku.
- `0x01` – wyczyszczenie całej pamięci ekranu oraz powrót na pozycję home.

- **Blok główny programu**

```
int main(void)
{
    LCD_inicjalizacja();

    //char tekst[5] = {'K', 'a', 'm', 'i', 'l'};
    LCDinstrukcja(0x01);
    _delay_ms(1);

    uint8_t wiersz = 0, kolumna = 0;

    LCDustawKursor(wiersz, kolumna);

    LCDtekst("Kamil Sitarski 123456", kolumna);

    while (1)
    {
    }
}
```

Po uruchomieniu mikrokontrolera sterowanie trafia do funkcji `main`. Pierwszym krokiem jest konfiguracja sprzętu za pomocą `LCD_inicjalizacja()`. Następnie programowo czyszczony jest ekran (`LCDinstrukcja(0x01)`) i następuje ustawienie kursora na logiczną pozycję startową (0, 0) (lewy górny róg ekranu). Wywołanie funkcji `LCDtekst` z argumentem ciągu znaków powoduje przesłanie napisu "Kamil Sitarski 123456". Po przetworzeniu tekstu i przejściu do nowej linii, mikrokontroler zostaje wprowadzony w nieskończoną, pustą pętlę `while(1)`, która utrzymuje stabilny stan pracy procesora i przeciwdziała samoczynnemu restartowi pętli głównej.

4. Kod

```
/*
 * Laboratorium6.c
 *
 * Created: 02.06.2026 16:24:32
 * Author : student_pl
 */

#define F_CPU 1000000UL
#define PORT_LCD PORTA
#define PIN_E PA0
#define PIN_RS PA1

#include <avr/io.h>
#include <util/delay.h>
#include <string.h>

void LCDwypiszStarszaCzesc(char bajt){
    PORT_LCD |= (1 << PIN_E);
    _delay_us(50);
    PORT_LCD = (bajt & 0xF0) | (PORT_LCD & 0x0F);
    _delay_us(50);
    PORT_LCD &= ~(1 << PIN_E);
}

void LCDwpisz(char bajt){
    LCDwypiszStarszaCzesc(bajt);
    _delay_us(50);
    PORT_LCD |= (1 << PIN_E);
    _delay_us(50);
    PORT_LCD = ((bajt & 0x0F) << 4) | (PORT_LCD & 0x0F);
    _delay_us(50);
    PORT_LCD &= ~(1 << PIN_E);
}

void LCDinstrukcja(char bajt){
    PORT_LCD &= ~(1 << PIN_RS);
    _delay_ms(1);
    LCDwpisz(bajt);
    _delay_ms(1);
    PORT_LCD |= (1 << PIN_RS);
}

void LCDustawKursor(uint8_t w, uint8_t k){
    PORT_LCD &= ~(1 << PIN_RS);
    uint8_t bajt = 0b10000000;
    if(w==1){
        bajt = 0b11000000;
    }
    bajt += k;
    LCDwpisz(bajt);
    _delay_ms(1);
    PORT_LCD |= (1 << PIN_RS);
}
```

```

void LCDtekst(char *tekst, uint8_t kolumna){
    for(uint8_t i=0 ; i<strlen(tekst);i++){
        if(kolumna==16){ LCDustawKursor(1,0); }
        _delay_ms(1);
        LCDwypisz(tekst[i]);
        kolumna++;
    }
}

void LCD_inicjalizacja(){
    DDRA = 0xFF;

    LCDwypiszStarszaCzesc(0x30);
    _delay_ms(5);
    LCDwypiszStarszaCzesc(0x30);
    _delay_ms(1);
    LCDwypiszStarszaCzesc(0x30);
    _delay_ms(1);
    LCDwypiszStarszaCzesc(0x20);

    _delay_ms(1);
    LCDwypisz(0x28);
    _delay_ms(1);
    LCDwypisz(0b00001100);//00001100
    _delay_ms(1);
    LCDwypisz(0x06);
    _delay_ms(1);
    LCDwypisz(0x01);
    _delay_ms(1);
    PORT_LCD |= (1 << PIN_RS);
}

int main(void)
{
    LCD_inicjalizacja();

    //char tekst[5] = {'K', 'a', 'm', 'i', 'l'};
    LCDinstrukcja(0x01);
    _delay_ms(1);

    uint8_t wiersz = 0, kolumna = 0;

    LCDustawKursor(wiersz, kolumna);

    LCDtekst("Kamil Sitarski 123456", kolumna);

    while (1)
    {
    }
}

```

5. Podsumowanie i wnioski

Realizacja ćwiczenia laboratoryjnego pozwoliła na praktyczne zapoznanie się z niskopoziomową obsługą alfanumerycznego wyświetlacza LCD ze sterownikiem HD44780 przy użyciu mikrokontrolera jednoukładowego AVR ATmega32. Na podstawie przeprowadzonych badań i wykonanego projektu programistycznego sformułowano następujące wnioski:

1. **Efektywność wykorzystania zasobów:** Zastosowanie interfejsu 4-bitowego zamiast standardowego trybu 8-bitowego pozwala na znaczną optymalizację wykorzystania linii wejścia/wyjścia mikrokontrolera. Redukcja liczby wymaganych przewodów danych z 8 do 4 umożliwia zaoszczędzenie cennych zasobów sprzętowych procesora, co ma kluczowe znaczenie w projektowaniu kompaktowych systemów wbudowanych.
2. **Krytyczne znaczenie reżimu czasowego:** Sterownik HD44780 jest układem o ściśle określonych wymaganiach dotyczących zwłok czasowych. Zaimplementowanie programowych opóźnień (rzędu mikrosekund przy strobnowaniu linią EN oraz milisekund przy inicjalizacji i czyszczeniu ekranu) jest bezwzględnym warunkiem poprawnej synchronizacji. Zbyt krótkie czasy oczekiwania prowadzą do gubienia paczek danych lub błędnej interpretacji rozkazów przez wyświetlacz.
3. **Złożoność procedury inicjalizacji:** Prawidłowe przetączenie kontrolera LCD w tryb 4-bitowy wymaga bezwzględnego przestrzegania sekwencji resetu programowego (trzykrotne wymuszenie zapisu wartości 0x30). Pominięcie tego kroku lub niewłaściwa konfiguracja rejestru wyboru instrukcji ($RS = 0$) uniemożliwia poprawny start peryferium.
4. **Programowa kontrola ograniczeń sprzętowych:** Ponieważ struktura pamięci DDRAM wyświetlacza nie pokrywa się bezpośrednio w sposób ciągły z fizycznymi wierszami matrycy, kluczowe staje się algorytmiczne kontrolowanie pozycji kursora. Wprowadzenie warunku sprawdzającego przepiętnienie kolumny pozwoliło na automatyczne, płynne przenoszenie strumienia danych tekstowych, co uniezależniło czytelność komunikatu od fizycznych ograniczeń rozmiaru ekranu